

ТЕХНИЧЕСКАЯ СТАТЬЯ

UniversalToolchain

Архитектура расширяемого DSL-runtime для .NET

Композиция диалектов, промежуточные представления, бэкенды
и верификация

Wist как эталонный язык и первый публичный facade

Разаков Михаил Русланович

Версия статьи: 1.0, 20 июля 2026 г.

Описываемая реализация: репозиторий Wist2 / UniversalToolchain

Содержание

Аннотация	1
Список сокращений	1
1 Назначение, область применения и статус	2
1.1 Модель пользователей	2
1.2 Проверяемые свойства	3
1.3 Статус доказательств	4
2 Соседние классы решений и граница применимости	4
3 Архитектура и границы ответственности	6
3.1 Канонический pipeline	7
3.2 Архитектурные границы	7
4 Модульный фронтенд	8
4.1 Расширение по стадиям	8
4.2 Контракты автора модуля	8
4.3 Анти-паттерн локального обходного пути	8
5 Bytecode, AIR и legality преобразований	9
5.1 Разделение представлений	9
5.2 Стековая и типовая дисциплина	10
5.3 Intrinsics и runtime calls	10
6 Диалекты, manifests и runtime composition	10
6.1 Диалект как конфигурация языка и runtime	10
6.2 Путь композиции	11
6.3 Детерминированность, groups и aliases	11
6.4 Runtime manifests, capabilities и ownership	12
7 Исполнение и backend-neutral lifecycle	13
7.1 Reference interpreter	13
7.2 CIL/DynamicMethod backend	13
7.3 Semantic parity	13
7.4 Compiled artifact и execution session	14
7.5 Нейтральный runtime host и ограничение расширяемости	14
8 Публичный Wist facade, диагностика и граница доверия	15
8.1 WistEngine и typed API	15
8.2 Restricted arithmetic preset	15
8.3 Options snapshot, profiles и custom dialect source	16
8.4 Structured diagnostics и preflight limits	16
8.5 Host-owned actions и безопасность	16
8.6 Versioned debug trace	18
9 Экспериментальный SSA-маршрут	18
9.1 Место SSA в системе	18

9.2	Политики запуска и failure semantics	19
9.3	Модель SSA и managed-call bindings	19
9.4	Pass pipeline и verifiers	19
9.5	Проекция Wist-операций и отчёт	19
10	Верификация архитектуры	20
10.1	Тесты как executable contracts	20
10.2	Positive, negative и differential coverage	20
10.3	Экспериментальные сценарии	21
10.4	Verification record и текущий snapshot	22
11	Методика измерения производительности	23
11.1	Разделение фаз	23
11.2	Compilation suite	23
11.3	Convenience suite	23
11.4	Hot-path suite	24
11.5	Evidence package и допустимые выводы	24
12	Текущее состояние, ограничения и направления развития	24
13	Личный вклад автора	25
14	Заключение	26
Приложение А.	Команды воспроизведения	26
A.1.	Идентичность исходного дерева	26
A.2.	Окружение и каноническая проверка	27
A.3.	Публичный sample и CLI trace	27
A.4.	Package surface и clean consumer	27
A.5.	Измерения производительности	27
Приложение Б.	Матрица доказательств	28
Приложение В.	Термины	28
Приложение Г.	Публичные контракты Wist facade	29
Г.1.	WistEngineOptions	29
Г.2.	WistValidationResult	30
Г.3.	WistProgram	30
Г.4.	WistCompileResult	30
Г.5.	WistOptimizationReport	30
Приложение Д.	Диалектные сценарии	30
Д.1.	Минимальная арифметика	31
Д.2.	Pricing-restricted	31
Д.3.	Function-calls-safe-math	31
Д.4.	SSA profile	31
Д.5.	Composition-restricted	31

Аннотация

UniversalToolchain — модульный .NET-фреймворк для построения и исполнения предметно-ориентированных языков. Wist служит эталонным языком проекта и одновременно предоставляет первый пользовательский слой в виде пакета UniversalToolchain.Wist. В статье описана текущая архитектура системы: модульный фронтенд, разделённые представления Bytecode и Abstract IR, диалектную композицию, manifest-backed выбор runtime-компонентов, capability-контракты, интерпретируемый и CIL-компилируемый пути, backend-neutral модель артефакта и сессии, публичный typed facade, структурированную диагностику, debug trace и экспериментальный SSA-маршрут.

Центральное требование проекта — сохранение явных границ между языком, конфигурацией runtime, преобразованиями IR и исполнением. Диалект должен завершить выбор компонентов до создания execution host; frontend-модули не должны зависеть от конкретного backend; backend-specific intrinsics допустимы только при объявленной capability; общая семантика интерпретатора и CIL-пути проверяется parity-тестами; изменяемое состояние запуска принадлежит session, а не переиспользуемому compiled artifact.

Статья сознательно отделяет реализованные свойства от желаемых. Restricted dialect ограничивает языковую поверхность, но не является процессной песочницей. SSA применяется только как opt-in verifier-gated маршрут для поддерживаемого подмножества. Производительность оценивается отдельно для компиляции, convenience API и горячего вызова заранее скомпилированного делегата. Численные показатели текущего snapshot не объявляются подтверждёнными без нового clean build/test/benchmark evidence package.

Ключевые слова: предметно-ориентированные языки, compiler framework, .NET, runtime composition, Bytecode, Abstract IR, CIL, backend capabilities, semantic parity, SSA, Wist.

Список сокращений

Сокращение	Значение
DSL	Domain-Specific Language, предметно-ориентированный язык
AST	Abstract Syntax Tree, абстрактное синтаксическое дерево
IR	Intermediate Representation, промежуточное представление
AIR	Abstract IR проекта UniversalToolchain
SSA	Static Single Assignment form
CIL	Common Intermediate Language платформы CLI/.NET

Сокращение	Значение
CLI	Common Language Infrastructure; в контексте утилиты также command-line interface, значение определяется контекстом
CLR	Common Language Runtime
DI	Dependency Injection
SCCP	Sparse Conditional Constant Propagation
API	Application Programming Interface

1. Назначение, область применения и статус

UniversalToolchain предназначен для случаев, когда приложению недостаточно статического C#-метода или простого evaluator, но полноценная language workbench-платформа избыточна. Типичный сценарий — формула или ограниченная вычислительная политика, которую требуется хранить отдельно от кода приложения, проверять до запуска, исполнять через выбранный backend и встраивать через типизированный API.

Проект разделён на три уровня ответственности:

- **UniversalToolchain** — общий framework: стадии компиляции, IR, manifests, capabilities, оптимизаторы и backend contracts;
- **Wist** — reference language и proving ground для проверки framework-механизмов на реальном синтаксисе;
- **UniversalToolchain.Wist** — first-contact NuGet package с WistEngine, restricted arithmetic preset, diagnostics и typed delegate API.

Прикладной host остаётся владельцем входных данных, авторизации, side effects, persistence, timeout, quotas и process isolation. Формула вычисляет значение; решение о действии принимает приложение. Это правило является частью архитектуры, а не рекомендацией по стилю.

Документ не описывает UniversalToolchain как замену C#, Roslyn, ANTLR, MLIR или MPS. Он фиксирует более узкий предмет: как организовать расширяемый DSL runtime в .NET так, чтобы состав языка, активированные runtime-компоненты и способ исполнения были детерминированными и проверяемыми [1, 2].

1.1. Модель пользователей

UniversalToolchain и Wist необходимо рассматривать на разных уровнях.

UniversalToolchain — framework-слой. Его потребителем является разработчик языка, модуля, оптимизатора или backend runtime. На этом уровне важны контракты стадий, manifests, capabilities, deterministic ordering и верификаторы.

Wist — эталонный язык и proving ground. Он объединяет поставляемые мо-

дули в работающие профили и позволяет проверять общие механизмы на реальном синтаксисе.

UniversalToolchain.Wist — first-contact NuGet package. Его потребитель не обязан понимать Bytecode, AIR или DI. Он создаёт WistEngine, проверяет формулу и при необходимости получает типизированный делегат.

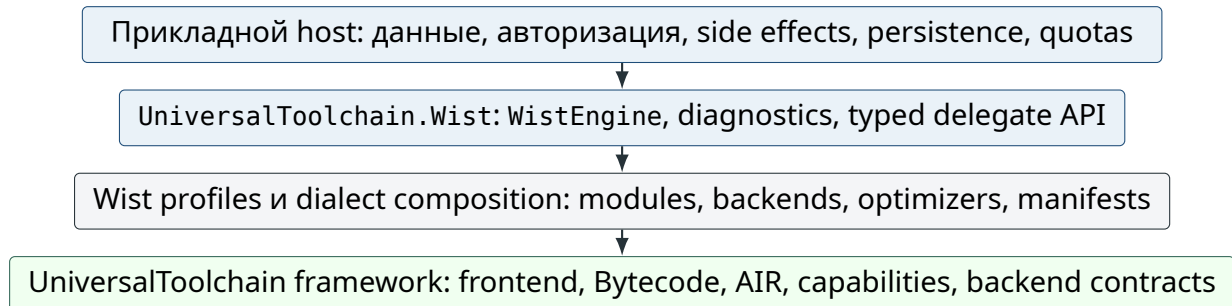


Рис. 1: Уровни проекта и направление зависимости пользовательского слоя

Таблица ниже связывает пользователей с реальной поверхностью проекта.

Пользователь	Задача	Основной вход	Необходимые знания
Разработчик приложения	встроить ограниченную числовую формулу	WistEngine	параметры, diagnostics, host security
Автор диалекта	выбрать языковые и runtime-возможности	.wistdialect	aliases, groups, backend и optimizer directives
Автор frontend-модуля	добавить синтаксис и lowering	module contracts	lexer, parser, AST и Bytecode ownership
Автор оптимизатора	преобразовать AIR или SSA	capability + verifier contracts	семантика IR и target legality
Автор backend	исполнить backend-neutral input	runtime manifests и artifact compiler	AIR, stack/type discipline, parity
Исследователь или студент	изучить compiler pipeline	профили Wist, trace и тесты	основные понятия компиляторов

1.2. Проверяемые свойства

Инвариант	Что защищает	Механизм
Frontend не зависит от backend	возможность нескольких способов исполнения	Bytecode/AIR contracts, project references, tests
Composition предшествует activation	отсутствие невыбранных компонентов	build plan и selected runtime plan

Инвариант	Что защищает	Механизм
Порядок детерминирован	воспроизводимость	явные aliases, sorting/ordering, conflict diagnostics
Общий язык имеет parity Intrinsic требует capability	единая семантика target legality	paired interpreter/CIL tests capability IDs и optimizer gating
Преобразование проверяется	корректность IR	AIR/SSA verifiers
Публичный API backend-neutral	стабильная интеграция	compiled artifact/session и facade
Restricted dialect не равен sandbox	корректная модель доверия	security docs, limits, host-owned actions

1.3. Статус доказательств

Описание основано на исследованном snapshot `Wist2(1).zip` и текущем дереве проекта [3]. Встроенный `VERIFICATION.md` относится к более раннему baseline: текущий snapshot содержит изменения после него и несовпадения recursive manifest для нескольких файлов. Поэтому в статье различаются три категории фактов:

1. свойства, подтверждённые непосредственной инспекцией текущего исходного дерева;
2. результаты, зафиксированные предыдущим verification record;
3. показатели, требующие нового clean-unpack build/test/pack/docs/benchmark прогона.

Такое разделение предотвращает перенос старого зелёного статуса на новый snapshot без повторной проверки.

2. Соседние классы решений и граница применимости

Обычный C# должен оставаться базовым вариантом сравнения. Если правило редко меняется, принадлежит команде разработки и не требует внешнего редактирования, отдельный DSL создаёт дополнительную стоимость: собственные диагностики, документацию, тесты, версионирование и поддержку. C# предоставляет полную модель типов, стандартный компилятор, отладчик и высокопроизводительное исполнение. UniversalToolchain не предназначен для вытеснения этого сценария.

Ограничение C# проявляется не в способности выразить формулу, а в чрезмерной ширине доступной среды, когда код поступает извне. Roslyn открывает объектную модель традиционного компиляторного конвейера и scripting API [4, 5]. Это мощная платформа для инструментов, анализа и выполнения C#-фрагментов. Однако приложению, которому нужен небольшой язык фор-

мул, должно самостоятельно проектировать allowlist, ссылки на сборки, модель доверия и ограничения контекста. Использование полноценного языка оправдано, когда требуются его семантика и инструменты; оно не является автоматически лучшим способом построить узкий domain language.

NCalc определяется авторами как evaluator математических выражений для .NET с параметрами и пользовательскими функциями [6]. Dynamic Espresso интерпретирует подмножество C#, создавая expression trees или делегаты [7]. Такие библиотеки удобны, когда продукту требуется компактное выражение и заранее известная модель интеграции. Их преимущество — низкий порог входа и малая архитектурная стоимость.

UniversalToolchain становится оправданным только после появления требований другого класса: профили языка, модульная синтаксическая и семантическая композиция, выбранный runtime plan, несколько бэкендов, backend capability checks, собственные IR и экспериментальные passes. Иными словами, проект не конкурирует с evaluator по минимальности API. Он исследует переход от evaluator к поддерживаемому DSL runtime.

Microsoft RulesEngine позиционируется как .NET rules engine с динамическими выражениями и workflow-моделью [8]. Движок правил решает задачу организации, выбора и выполнения бизнес-правил. UniversalToolchain имеет другой центр тяжести: язык, стадии lowering, runtime composition, оптимизаторы и backend-neutral execution. Wist может применяться как вычислительная часть правила, но текущий публичный продукт сознательно не владеет действиями. Формула возвращает данные, а host application решает, менять ли состояние, выполнять ли операцию и какие права для этого нужны.

Такое разделение предотвращает смешивание двух задач: «вычислить ограниченное выражение» и «оркестрировать жизненный цикл бизнес-правил». В текущем snapshot прежний специальный rules-layer удалён; нормальные Wist-конструкции и facade не должны скрыто восстанавливать его ответственность.

ANTLR генерирует lexer/parser и parse tree walker из грамматики [9]; алгоритмическая основа семейства ANTLR описана в работе об LL(*) [10]. Генератор парсеров закрывает важную, но ограниченную часть задачи. После построения дерева остаются binding, IR, проверка типов и стека, исполнение, оптимизации, диагностика, runtime composition и security boundary.

UniversalToolchain использует собственную модульную модель фронтенда. Это не означает, что собственный parser принципиально лучше генератора. Выбор связан с исследованием расширения по стадиям и существующей архитектурой Wist. В другом проекте ANTLR может быть правильным frontend-компонентом внутри аналогичного runtime pipeline.

JetBrains MPS является полноценной средой языковой инженерии. Projectional editor работает непосредственно с AST и позволяет задавать различные проекции языка [11]. Такой подход предоставляет мощные средства композиции языков, редакторов и генераторов. Для узкого runtime, встраиваемого в .NET-приложение, это иной масштаб решения: MPS владеет

процессом разработки языка и редактором, тогда как UniversalToolchain сосредоточен на библиотечном execution stack и типизированной интеграции с хостом.

MLIR предлагает расширяемую инфраструктуру с dialect-механизмом и несколькими уровнями абстракции [12, 13]. Он демонстрирует общий принцип: разные domain- и target-представления могут сосуществовать и понижаться через формализованные преобразования. UniversalToolchain существенно меньше и не пытается повторить MLIR. Сходство находится на уровне архитектурной идеи — промежуточное представление должно иметь явную семантику и владельца, а расширение не должно сводиться к произвольным строковым тегам.

Truffle исследует другой путь: язык реализуется как AST interpreter, который специализируется и частично вычисляется в оптимизированный код [14]. В UniversalToolchain интерпретатор сознательно остаётся ограниченным reference backend, а быстрый путь строится через AIR и CIL. Это решение проще для текущей цели, но не обладает свойствами self-optimizing runtime.

Класс решения	Сильная сторона	Что остаётся за пределами	Позиция проекта
Обычный C#	типизация, инструменты, производительность	внешнее ограниченное редактирование	использовать по умолчанию, если DSL не нужен
NCalc / Dynamic Espresso	компактное вычисление выражений	полный lifecycle собственного DSL runtime	проще Wist для небольших формул
RulesEngine	организация и выполнение правил	compiler framework и backend research	соседний продуктовый класс
ANTLR	генерация lexer/parser	lowering, runtime, бэкенды	потенциальный frontend-инструмент
MPS	комплексная языковая инженерия и редактор	лёгкая библиотечная интеграция	значительно более тяжёлая платформа
MLIR	многоуровневая extensible IR infrastructure	готовый пользовательский .NET formula API	концептуальный ориентир, не прямой аналог
Проект	runtime composition, IR, два бэкенда, typed facade	workbench, sandbox и широкая апробация	прототип framework

3. Архитектура и границы ответственности

3.1. Канонический pipeline

Канонический путь UniversalToolchain представлен на рисунке. Разделение синтаксического анализа, промежуточных представлений, оптимизаций и target lowering соответствует базовой архитектуре компиляторных систем [15, 16].

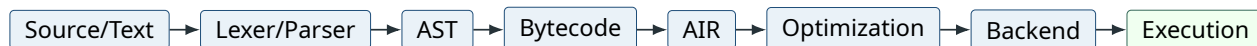


Рис. 2: Канонический pipeline UniversalToolchain

Каждый уровень отвечает на собственный вопрос:

- **Source/Text:** какой исходный текст передан и какая допустима предобработка;
- **Lexer/Parser/AST:** как текст распознан как синтаксическая структура;
- **Bytecode:** как вклады независимых frontend-модулей объединены в нормализованный пост-AST слой;
- **AIR:** как семантика представлена для исполнения и преобразований независимо от исходного синтаксиса;
- **Optimization:** какие преобразования доказуемо допустимы при выбранных target capabilities;
- **Backend:** как AIR выполняется интерпретатором или понижается к CIL;
- **Execution:** как compiled artifact получает конкретные значения параметров и возвращает результат.

Наличие нескольких уровней увеличивает объём кода по сравнению с AST evaluator. Это осознанная цена за независимость фронтенда от бэкенда. Если arithmetic AST node непосредственно эмитирует CIL, добавить интерпретатор или другой backend можно только путём дублирования frontend-семантики. Если backend анализирует исходные AST-типы, он становится зависимым от языка. Bytecode и AIR разрывают эту связь. Подобное разделение не требует копировать конкретную внешнюю инфраструктуру, но следует общему принципу: представление должно соответствовать классу анализов и преобразований, для которых оно предназначено [17, 18].

3.2. Архитектурные границы

В проекте различаются четыре вида границ.

1. **Синтаксическая граница** отделяет распознавание текста от семантического lowering.
2. **IR-граница** отделяет вклад language feature от backend implementation.
3. **Композиционная граница** отделяет описание желаемых возможностей от активации runtime-объектов.
4. **Публичная граница** отделяет типизированный API приложения от внутренних артефактов компилятора.

Нарушение каждой границы имеет характерный симптом. Backend-specific имя в frontend-модуле нарушает IR-границу. Создание всех зарегистрирован-

ных бэкендов до выбора диалекта нарушает композиционную границу. Возврат `DynamicMethod` из публичного API нарушает публичную границу. Изменение AST дальнего предка внутри локального `parser callback` нарушает синтаксическое владение.

4. Модульный фронтенд

4.1. Расширение по стадиям

Модульность `UniversalToolchain` не сводится к регистрации класса в DI. Frontend-модуль участвует только в нужных стадиях: может добавить `token pattern`, `parser node`, `AST visitor`, `lowering` в `Bytecode` или постобработку. Числовой литерал не должен знать о циклах; арифметика не должна зависеть от CIL; `whitespace module` не должен владеть `runtime state`.

Такое разделение уменьшает прямые зависимости, но создаёт скрытые контракты. Строковое имя токена связывает `lexer` и `parser`. Приоритет `parser rule` определяет владение неоднозначной грамматикой. Несколько `visitors` могут получить один узел. `Bytecode tag` может иметь производителя и нескольких потребителей. `Mutable state visitor`-а может протечь между компиляциями. Поэтому интерфейс расширения сам по себе недостаточен: нужны правила `ownership` и тесты отрицательных случаев.

4.2. Контракты автора модуля

Контракт	Риск без контракта	Требуемая проверка
Имена токенов	<code>lexer</code> и <code>parser</code> используют разные строковые ключи	единый <code>catalog/feature test</code>
Приоритеты грамматики	новая <code>rule</code> перехватывает существующий синтаксис	<code>parser ownership tests</code>
Владение <code>AST node</code>	несколько <code>visitors</code> генерируют дублирующий код	фильтрация входа и <code>processed-state tests</code>
<code>Bytecode tags</code>	теги превращаются в неописанный второй язык	<code>producer/consumer semantics</code> и <code>verifier roadmap</code>
Состояние компиляции	данные одной программы влияют на другую	<code>execution-scoped ownership</code>
Диалектная включаемость	<code>feature</code> нельзя надёжно отключить	<code>positive/negative dialect tests</code>
<code>Backend capability</code>	frontend создаёт неподдерживаемый <code>intrinsic</code>	<code>capability-gated lowering</code>

4.3. Анти-паттерн локального обходного пути

Предположим, требуется ускорить сложение целых чисел. Наиболее короткое изменение — в `arithmetic module` сразу сгенерировать CIL-specific

инструкцию. Успешный пример станет быстрее, но возникнут четыре дефекта: `interpreter` перестанет понимать программу; `frontend` будет знать `target`; `restricted profile` не сможет корректно описать потребность; оптимизация окажется обязательной частью семантики. Правильный путь длиннее: `feature lowering` создаёт нейтральное представление; `optimizer` распознаёт поддерживаемый `pattern`; `backend` объявляет `capability`; только после этого операция может быть заменена на `typed intrinsic`.

Этот пример иллюстрирует общий принцип проекта: точка расширения выбирается по владельцу семантики, а не по месту, где проще всего вставить условие.

5. Bytecode, AIR и legality преобразований

5.1. Разделение представлений

AST отражает структуру исходной программы и сохраняет связь с синтаксисом. Для `parser` и диагностик это преимущество, однако непосредственное исполнение AST плохо масштабируется при появлении модулей, нескольких `backend`-путей и оптимизаций. UniversalToolchain вводит два последующих слоя: Bytecode и AIR.

Bytecode является границей, на которой вклад `frontend`-модулей объединяется после AST. Он удобен для последовательного `lowering` конструкций языка, меток, стека и семантических аннотаций. Bytecode ещё близок к тому, как `language features` строят программу.

AIR предназначен для оптимизаторов и бэкендов. Он должен описывать исполняемую семантику без необходимости восстанавливать исходный `parser node`. AIR является владельцем `core opcodes`, `intrinsics`, типов операндов, переходов и метаданных, необходимых для проверки и `lowering`.

Представление	Главный потребитель	Что сохраняет	Что не должно знать
AST	<code>parser</code> и <code>syntax visitors</code>	структуру исходного языка	CIL и <code>backend-specific intrinsics</code>
Bytecode	<code>frontend lowering</code>	нормализованный вклад модулей	детали конкретного <code>emitter-a</code>
AIR	<code>optimizers</code> и <code>backends</code>	исполняемую семантику, <code>control/data operations</code>	исходные <code>parser types</code>
CIL artifact	CLR	конкретный исполняемый код	язык, из которого он получен

Разделение полезно только при соблюдении семантических контрактов. Если Bytecode является произвольным набором строковых `tags`, а AIR лишь копирует эти `tags`, граница формальна. Поэтому проект постепенно переводит

неявные соглашения в catalogs, readers, capabilities и verifiers. Технический долг в части полного реестра bytecode tags прямо зафиксирован в ограничениях.

5.2. Стековая и типовая дисциплина

Для stack-oriented представления недостаточно проверить число инструкций. Каждая операция имеет входной и выходной эффект на стек. Переходы должны соединять совместимые состояния, значение не должно читаться до помещения, а backend lowering обязан сохранять типовую семантику. Ошибка в этой области может проявиться только на одной ветви или после оптимизации, поэтому happy-path test недостаточен.

В проекте stack discipline защищается несколькими уровнями: контрактами инструкций, проверками AIR, parity-тестами и backend-specific validation. Для SSA-route стековый AIR сначала переводится в граф значений; перед lowering и после обратной emission выполняется отдельная проверка. Это снижает риск, что оптимизация корректна в SSA-модели, но возвращает несовместимый stack program.

5.3. Intrinsics и runtime calls

AIR поддерживает понятие intrinsic — операции, которую backend может реализовать более прямым способом. Однако intrinsic не должен становиться скрытым требованием ко всем бэкендам. Текущий interpreter сохраняется как минимальный reference path: он исполняет core control-flow/data opcodes и универсальные вызовы call C# / call C# ctor. Feature-specific или оптимизированные load_*, add_*, cmp_* и подобные intrinsics не включаются в интерпретатор только ради прохождения теста.

Если feature должна работать в reference backend, она понижается к обычному разрешённому runtime-вызову. Если CIL backend поддерживает более быстрый intrinsic, optimizer может заменить pattern только после проверки capability. Такая политика сохраняет различие между семантикой языка и ускорением конкретной цели.

6. Диалекты, manifests и runtime composition

6.1. Диалект как конфигурация языка и runtime

В UniversalToolchain диалект описывает выбранный вариант языка и среды выполнения. Он может включать frontend-модули, группы возможностей, бэкенды, оптимизаторы и ограничения. Поэтому два профиля с одинаковым синтаксисом, но разными backend/optimizer settings, являются разными execution configurations.

Поставляемый snapshot содержит девять .wistdialect-профилей:

- full-default;

- full-default-native;
- minimal-arithmetic;
- minimal-arithmetic-native;
- minimal-arithmetic-grouped;
- pricing-restricted;
- composition-restricted;
- function-calls-safe-math;
- ssa.

Наличие профиля в репозитории не означает одинаковую зрелость всех возможностей. `pricing-restricted` и `restricted arithmetic facade` демонстрируют узкий пользовательский сценарий; `ssa` является экспериментальным opt-in профилем.

6.2. Путь композиции

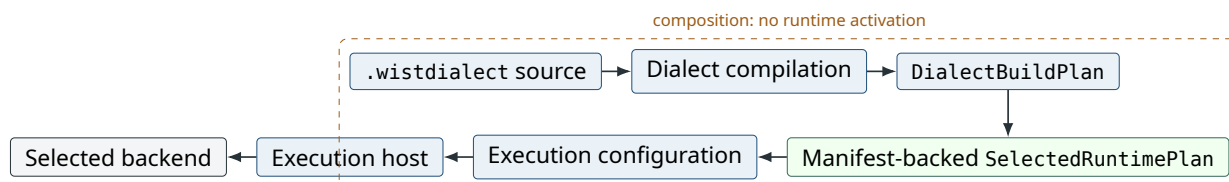


Рис. 3: Композиция диалекта и выбор runtime-компонентов до activation

Исходное описание диалекта компилируется в `DialectBuildPlan`. Plan хранит требования в явном порядке: выбранные модули, backend directives, optimizer directives, groups и capability expectations. Затем resolver сопоставляет требования с manifest-backed каталогом и строит `SelectedRuntimePlan`. Только после успешного разрешения создаётся execution host.

Это разделение принципиально. Если DI container сначала создаёт все зарегистрированные runtime-компоненты, а затем фильтрует ненужные, ограниченный профиль уже не контролирует активацию. В текущей модели composition и activation разделены: каталог описывает доступное, plan — требуемое, selected plan — фактически разрешённое.

6.3. Детерминированность, groups и aliases

Детерминированность означает не просто одинаковый итоговый числовой результат. Для одного source dialect и одного набора manifests должны совпадать:

- порядок выбранных модулей;
- идентичность бэкендов;
- набор оптимизаторов;
- диагностические коды и порядок конфликтов;
- capability map;
- поведение при отсутствии обязательного компонента.

Resolver не должен полагаться на порядок обхода файловой системы,

reflection enumeration или регистрации service collection. Там, где порядок семантически важен, он задаётся plan-ом; там, где несколько элементов равноправны, применяется стабильная сортировка по идентификатору/alias. Отсутствующие descriptor-ы и несовместимости становятся диагностикой composition, а не поздним NullReferenceException во время исполнения.

Человеку неудобно перечислять десятки низкоуровневых компонентов. Поэтому диалекты могут использовать группы и aliases. Однако удобное имя не должно становиться неявной глобальной истиной. Group expansion выполняется в контексте известного каталога, а результат снова превращается в явный ordered plan. Alias служит интерфейсом манифеста и должен быть стабилен в пределах заявленного контракта.

6.4. Runtime manifests, capabilities и ownership

Runtime manifest отделяет метаданные компонента от его активации. В manifest указываются идентификатор, alias, тип компонента, предоставляемые или требуемые capabilities и дополнительные свойства, необходимые resolver-у. Благодаря этому выбор может быть выполнен без создания всех типов и без знания конкретных class names в generic framework layer.

Manifest не является абсолютной security boundary. Он контролирует композицию, но runtime type resolution всё ещё требует корректной реализации и bounded reflection. Проект допускает reflection как ограниченный детерминированный механизм, а не как широкое автоматическое сканирование всех сборок.

Capability отвечает на вопрос: «может ли выбранная среда корректно выполнить данный контракт?». Например, optimizer, создающий typed arithmetic intrinsic, должен потребовать capability соответствующего backend. Наличие класса в assembly не является доказательством поддержки; имя backend в условии if также не является переносимым контрактом.

Capability-модель уменьшает распространение конкретных имён. Frontend генерирует нейтральную семантику. Optimizer объявляет требование. Backend manifest сообщает поддержку. Resolver и pipeline соединяют их. Если capability отсутствует, допустимы только заранее определённые исходы: преобразование не применяется, composition/compilation отклоняется или используется документированный fallback. Молчаливое создание неподдерживаемого IR запрещено.

Разрешение вызовов .NET-методов связано с потенциально широкой поверхностью выполнения. Allowlist провайдеров должен принадлежать политике композиции и runtime, а не выводиться из уже оптимизированного AIR. Иначе программа могла бы сначала сформировать запрос к произвольному provider, а затем легализовать его фактом присутствия в IR. Текущая архитектурная политика оставляет авторизацию во владении composition-слоя: AIR запрашивает вызов, но не назначает себе право на provider.

7. Исполнение и backend-neutral lifecycle

7.1. Reference interpreter

Интерпретатор используется как семантически простой путь для общего поддерживаемого подмножества. Его задача — не выиграть benchmark, а выполнять нейтральные core operations и универсальные managed calls. Чем меньше feature-specific логики находится в interpreter, тем легче использовать его как oracle для parity.

Это не означает, что interpreter автоматически всегда прав. Семантическое правило задаётся языковым контрактом и тестами. Однако при расхождении оптимизированный CIL path проверяется первым, поскольку он содержит больше lowering и target-specific решений.

7.2. CIL/DynamicMethod backend

CIL backend преобразует AIR в Common Intermediate Language, определённый стандартом ECMA-335 [19]. Для динамического исполняемого артефакта используется System.Reflection.Emit.DynamicMethod: .NET API представляет метод, который можно скомпилировать, выполнить и затем освободить [20]. Создание делегата является основным быстрым путём повторного вызова; Microsoft отдельно указывает, что вызов через delegate эффективнее reflection-style Invoke [21].

Архитектура проекта не утверждает генерацию собственного x64-кода. Wist эмитирует CIL, а дальнейшая JIT-компиляция и машинные оптимизации принадлежат CLR. Это различие важно для корректного описания результата: проект управляет frontend/lowering и CIL generation, но не заменяет .NET runtime.

7.3. Semantic parity

Semantic parity означает одинаковый наблюдаемый результат для программы, которая входит в общий контракт обоих бэкендов. Наблюдение может включать возвращаемое значение, разрешённые внешние вызовы, порядок эффектов и диагностируемый отказ. Parity не требует, чтобы внутренние инструкции или производительность совпадали.

Проверка строится попарно:

1. одна source program и одинаковые declared bindings;
2. один диалект, разрешающий оба backend-пути;
3. независимая компиляция/подготовка для каждого backend;
4. одинаковые входные значения;
5. сравнение результата или нормализованной ошибки.

Если feature реализована только в одном backend, она не включается в общий parity contract. Вместо ложного сравнения фиксируется capability boundary.

7.4. Compiled artifact и execution session

Без общей границы CIL compiler мог бы возвращать `DynamicMethod`, `interpreter` — список AIR instructions, а `public facade` вынужден был бы содержать две несовместимые ветви. Это связывает прикладной API с внутренней реализацией и усложняет третий backend.

UniversalToolchain вводит две сущности:

- `ICompiledArtifact` — неизменяемый результат компиляции с `source/binding metadata`;
- `ICompiledArtifactSession` — конкретный запуск с `mutable` значениями аргументов и `execution state`.

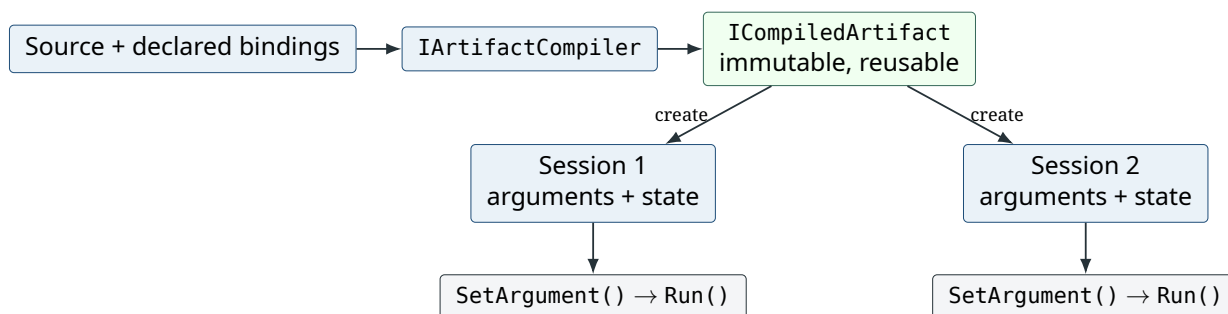


Рис. 4: Backend-neutral lifecycle: переиспользуемый артефакт и независимые execution sessions

Разделение артефакта и сессии также решает lifecycle-проблему. Скомпилированное представление может быть переиспользовано, но значения параметров и runtime context не должны протекать между независимыми запусками. Session является владельцем изменяемого состояния.

7.5. Нейтральный runtime host и ограничение расширяемости

`ToolchainRuntimeHost` выполняет `selected runtime configuration` через нейтральные `IArtifactCompiler` / `ICompiledArtifact` contracts. Wist execution host остаётся удобной обёрткой для `reference language`, но новое `generic runtime behavior` должно добавляться в нейтральный слой, а не в Wist-specific facade. Это предотвращает ситуацию, когда `framework` формально общий, но каждый `execution path` требует ветвления по Wist class names.

Наличие интерфейса не доказывает лёгкость добавления нового backend. Третий серьёзный backend потребует реализации `lowering`, `capabilities`, `manifests`, `parity matrix` и диагностики. Текущая граница снижает связанность и определяет место интеграции, но не заменяет фактический эксперимент с независимым target.

8. Публичный Wist facade, диагностика и граница доверия

8.1. WistEngine и typed API

Низкоуровневый framework содержит десятки проектов и типов, которые не должны быть первым контактом пользователя. Публичный пакет UniversalToolchain.Wist версии 0.1.0-alpha.1 вводит узкий facade для ограниченных формул. Основной класс — WistEngine.

Публичные операции:

Операция	Назначение	Исключения как основной результат
Validate(...)	проверить исходный текст и bindings без запуска	нет, возвращается WistValidationResult
Evaluate<T>(...)	разово подготовить и вычислить	возможно для programming/runtime errors
Compile<TDelegate>(...)	получить типизированную программу	throwing API для явного failure path
TryCompile<TDelegate>(...)	не бросать исключение при ожидаемой ошибке формулы	возвращается structured result

8.2. Restricted arithmetic preset

WistEngine.CreateRestrictedArithmetic() создаёт snapshot options и собирает ограниченный профиль. Он предназначен для числовой логики, а не для произвольных statements. Пример применения:

```

1 using UniversalToolchain.Wist;
2
3 using var engine = WistEngine.CreateRestrictedArithmetic();
4
5 const string formula =
6     "usage * 0.7 + reliability * 0.3 - incidents * 15.0";
7
8 var validation = engine.Validate(
9     formula,
10    new { usage = 100.0, reliability = 90.0, incidents = 1.0 });
11
12 if (!validation.IsValid)
13     throw new InvalidOperationException(validation.Message);
14
15 var score = engine.Compile<Func<double, double, double, double>>(
16     formula,
```

```
17     "usage",  
18     "reliability",  
19     "incidents");  
20  
21 double value = score.CompiledDelegate(100.0, 90.0, 1.0);
```

Тип делегата задаёт количество и CLR-типы параметров, а имена связывают их с `identifiers` формулы. `Compile-time generic constraint` гарантирует, что вызывающая сторона получает ожидаемый `delegate type`. Внутренняя компиляция остаётся скрыта в `WistProgram<TDelegate>`.

8.3. Options snapshot, profiles и custom dialect source

Помимо `preset`, `options` позволяют выбрать `backend`, `allowlist assemblies`, `resource limits`, `optimization settings` и `custom dialect source`, в том числе из текста. `Options snapshot`-ятся при создании `engine`, поэтому последующее изменение исходного объекта не должно неявно менять уже собранный `runtime`.

8.4. Structured diagnostics и preflight limits

Пользовательский API не должен заставлять приложение разбирать текст исключения. `WistDiagnostic` включает код, `severity`, `stage`, `source span`, `message` и `hints`. Эта форма позволяет UI подсветить участок формулы, отделить `validation error` от `composition/compilation failure` и сохранять стабильную машинную структуру при улучшении формулировок.

Диагностика особенно важна для `non-throwing API`. `Validate` и `TryCompile` возвращают объект результата; ожидаемая ошибка внешнего ввода не должна выглядеть как сбой приложения. Одновременно неожиданный `internal defect` не маскируется как обычная «неподдерживаемая формула».

`WistResourceLimits` ограничивает длину `source` и число параметров до тяжёлых стадий. Эти проверки защищают хост от очевидно чрезмерного ввода и дают предсказуемую диагностику. Они не ограничивают время CPU, объём памяти после запуска или внешние эффекты разрешённых вызовов. Поэтому называются `preflight limits`, а не `sandbox`.

`Allowlist` сборок ограничивает пространство `managed calls`, доступных `runtime composition`. Коллекция валидируется и `snapshot`-ится. Тем не менее разрешение сборки не эквивалентно разрешению каждого опасного поведения внутри неё; для `hostile input` необходим более узкий `provider policy` и процессная граница.

8.5. Host-owned actions и безопасность

Формула должна возвращать данные, а не владеть бизнес-действием. В примере `scoring result` может использоваться для решения «включить ли интерфейс», но само включение выполняет `host`. Это разделяет вычислитель-

ную семантику и права на side effects. Даже если язык в будущем получит дополнительные функции, авторизация, persistence и внешние операции не должны становиться случайной ответственностью evaluator.

Ограниченный диалект уменьшает класс допустимых программ. Например, restricted arithmetic принимает выражение и отклоняет statement-style форму с let. Это полезное свойство: неподдерживаемая конструкция не должна просачиваться через общий fallback.

Однако язык остаётся частью процесса приложения. Ошибка runtime, бесконечный или слишком дорогой вычислительный путь, разрешённый вызов с побочным эффектом и уязвимость хоста не устраняются фактом ограниченной грамматики.

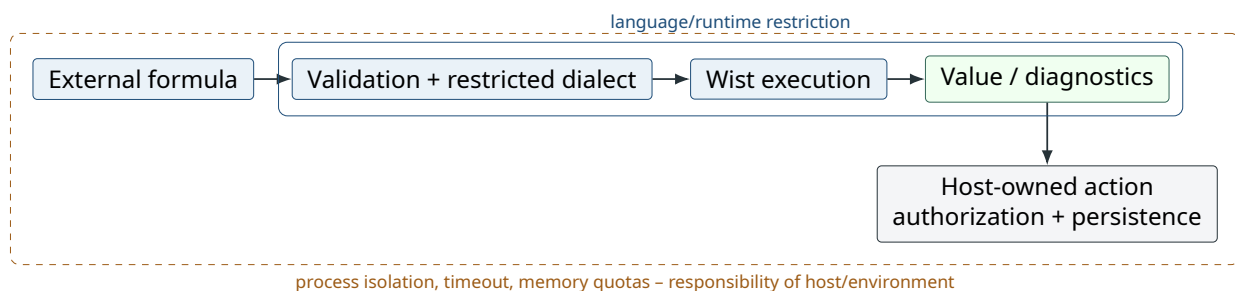


Рис. 5: Граница доверия: restricted language surface не заменяет внешнюю изоляцию

Риск	Что делает Wist	Что обязан сделать host/окружение
Слишком широкая грамматика	выбирает restricted dialect	определить допустимый профиль продукта
Некорректная формула	validation и diagnostics	не запускать invalid input
Чрезмерный source/parameters	preflight limits	установить значения по threat model
Опасные managed calls	allowlist/provider policy	минимизировать разрешённую поверхность
CPU/memory DoS	не гарантирует полное ограничение	процессная изоляция, quotas, timeout
Побочные эффекты	facade возвращает данные	host владеет action и authorization
Секреты в trace	redaction by default	политика хранения и доступа к артефактам

Нельзя утверждать, что CreateRestrictedArithmetic() создаёт безопасную песочницу для произвольного недоверенного кода. Корректная формулировка: preset ограничивает языковую и runtime-поверхность; hostile input требует отдельной изоляции процесса, лимитов ресурсов и явной модели угроз.

8.6. Versioned debug trace

Ранняя версия проекта содержала LogsViewer, отображавший source, lexemes, AST и Bytecode. По мере развития он перестал соответствовать фактическому pipeline: не показывал AIR, selected runtime plan, capabilities, SSA route и backend artifacts. Сохранение такого инструмента создавало бы ложное ощущение полной наблюдаемости. Поэтому legacy LogsViewer, ExecutorLoggerModule и logs.txt удалены.

Текущий CLI поддерживает `wistc run --trace trace.json`. Артефакт имеет versioned JSON shape, по умолчанию редактирует исходный текст и runtime values и содержит source length/hash, dialect/backend metadata, coarse stages, result/error summary.

Trace v2 является первой фазой, а не полным compiler explorer. Он подтверждает выбранную конфигурацию и крупные стадии, но пока не включает детальные dumps lexer, parser, AST, Bytecode, AIR, SSA и CIL. Статья сознательно не представляет эти будущие возможности как реализованные.

Профессиональная наблюдаемость требует не максимального количества логов, а стабильной схемы, redaction policy и связи каждого поля с владельцем. Сырые ToString() внутренних объектов непригодны как публичный формат: они меняются при refactoring и могут раскрыть исходные данные.

9. Экспериментальный SSA-маршрут

9.1. Место SSA в системе

Stack-oriented AIR удобен для исполнения и backend lowering, но некоторые глобальные преобразования естественнее выражаются в SSA, где каждое значение определяется один раз, а зависимости представлены явно. Классическая SSA-форма и алгоритмы размещения ф-функций описаны Cytron и соавторами [22]. Sparse conditional constant propagation объединяет распространение констант и достижимость ветвей [23].

В UniversalToolchain SSA не заменяет AIR и не является новым обязательным backend. Реализован экспериментальный маршрут:

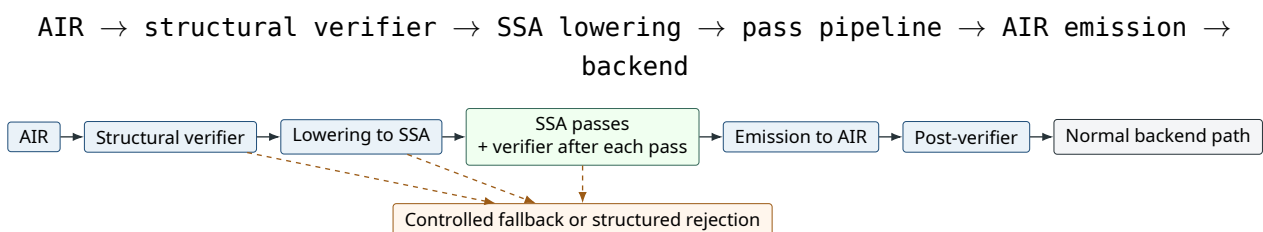


Рис. 6: Verifier-gated маршрут AIR – SSA – AIR

Такое положение SSA ограничивает риск. Пользовательский facade и backend contracts продолжают работать с существующей моделью, а эксперимент можно включить отдельно и наблюдать через структурированную диагностику и trace.

9.2. Политики запуска и failure semantics

WistSsaPolicy содержит четыре значения:

- Disabled — маршрут не используется;
- Prefer — маршрут применяется к поддерживаемому shape, а известная несовместимость может дать controlled fallback;
- Require — отсутствие возможности пройти маршрут считается ошибкой;
- Debug — маршрут используется с расширенной диагностикой и trace.

Семантика Prefer особенно важна. Fallback разрешён только для известных unsupported-route diagnostics. Неожиданный defect optimizer-а не должен маскироваться как «SSA не подошёл»; он завершается структурированной ошибкой UTC-WIST-SSA-001.

9.3. Модель SSA и managed-call bindings

Экспериментальная SSA-модель использует immutable blocks, block arguments, typed values, callable descriptors, facts, effects и детерминированные descriptor snapshots. Managed calls переносят точные execution-scoped bindings типов MethodInfo и ConstructorInfo через lowering, passes и emission. Production path не должен заново искать методы через AppDomain, Type.GetType или обход файловой системы. Такое rediscovery нарушило бы воспроизводимость и ownership разрешений.

9.4. Pass pipeline и verifiers

В текущем route реализованы:

- локальное constant folding;
- SCCP-lite;
- branch folding и очистка недостижимых блоков;
- удаление мёртвых чистых инструкций.

Число passes не является показателем зрелости. Для каждого преобразования важны предусловия, effect model, сохранение типов и проверка результата. После каждого pass выполняется verifier. Если pass выдаёт некорректный граф, pipeline останавливается на первой нарушенной границе.

9.5. Проекция Wist-операций и отчёт

Поддерживаемая арифметика Wist может проектировать native int32 add/subtract/multiply calls на канонические SSA callables. Это позволяет оптимизировать не только искусственный внутренний пример, но и реальный Wist route. При этом произвольные CLR values и общий scheduling SSA намеренно ограничены. Расширение coverage требует differential/oracle tests, а не только добавления converter branch.

WistOptimizationReport фиксирует:

- факт использования маршрута;
- controlled fallback;

- профиль;
- число входных и выходных AIR-инструкций;
- выполненные passes;
- diagnostics;
- debug trace для соответствующего режима.

Отчёт не доказывает ускорение. Он доказывает, какой маршрут реально был выполнен. Performance claim требует отдельного benchmark с одинаковой семантикой и raw artifacts.

10. Верификация архитектуры

10.1. Тесты как executable contracts

Для compiler framework количество тестов само по себе малоинформативно. Один и тот же happy path может быть покрыт сотнями параметров, в то время как неверный module order или fallback останется незамеченным. Поэтому тестовая стратегия проекта классифицируется по защищаемому риску.

Класс тестов	Защищаемый риск
Lexer/parser tests	неправильное распознавание, precedence и ownership
Module tests	скрытая зависимость feature от другого модуля
Dialect tests	неработающее включение/исключение возможности
Runtime selection tests	активация невыбранных компонентов, nondeterminism
Parity tests	расхождение interpreter и CIL
Negative restricted tests	просачивание запрещённого синтаксиса через fallback
Artifact/session tests	state leakage между запусками
Capability tests	backend-specific intrinsic в неподдерживаемом target
SSA route tests	invalid IR, неверный fallback, потеря binding/effect
Package surface tests	утечка внутренних implementation assemblies в compile-time API
Documentation smoke tests	устаревшие runnable examples
Benchmark prechecks	сравнение разных вычислений под одним названием

10.2. Positive, negative и differential coverage

Положительный тест показывает, что разрешённая возможность работает. Отрицательный тест доказывает, что граница действительно существует. Для restricted arithmetic недостаточно проверить $a + b$; необходимо отклонить let,

control flow и иной statement-style input. Для backend capability недостаточно успешно сгенерировать intrinsic в CIL; нужно проверить, что interpreter profile не получает эту инструкцию. Для SSA Prefer недостаточно увидеть fallback; неожиданный exception обязан не превращаться в fallback.

Interpreter и CIL используются как взаимные oracle на общем подмножестве. Набор программ должен включать:

- арифметические выражения и precedence;
- параметры разных допустимых типов;
- ветвления и сравнения, если доступны профилю;
- managed calls, входящие в allowlist;
- ошибки binding и unsupported feature;
- граничные значения, деление и преобразования типов;
- повторные сессии одного compiled artifact.

Сравнивается не только значение, но и тип результата, порядок разрешённых эффектов и нормализованная категория отказа.

10.3. Экспериментальные сценарии

Цель: проверить, что restricted profile принимает допустимую формулу и отклоняет более широкую форму до исполнения.

Вход A1:

```
1 usage * 0.7 + reliability * 0.3 - incidents * 15.0
```

Вход A2:

```
1 let score = usage * 0.7
2 score
```

Процедура: создать restricted arithmetic engine, передать одинаковый набор sample arguments в Validate, затем компилировать только A1.

Ожидаемый результат: A1 получает valid result и typed delegate; A2 получает structured validation failure. Ни один denied statement не должен быть исполнен через альтернативный route.

Цель: проверить H1.

Процедура:

1. несколько раз скомпилировать одинаковый dialect source при неизменном catalog;
2. сериализовать нормализованную проекцию DialectBuildPlan;
3. отдельно сериализовать SelectedRuntimePlan;
4. изменить порядок физического перечисления manifests, не меняя идентификаторы;
5. ввести отсутствующий backend alias и конфликтующего provider-a.

Метрики: ordered component IDs, capability map, diagnostic codes, отсутствие host creation при failure.

Ожидаемый результат: порядок физического обнаружения не влияет на selected plan; ошибочные входы завершаются стабильной composition diagnostic.

Цель: проверить H2.

Процедура: определить corpus программ общего подмножества; для каждой программы независимо подготовить interpreter и CIL artifact; выполнить на одинаковом наборе входов; сравнить значения, типы и нормализованные ошибки.

Критерий успеха: отсутствие различий на заявленном общем contract. Feature, не поддерживаемая одним backend, исключается из parity corpus и фиксируется в capability matrix.

Цель: проверить отсутствие state leakage.

Процедура: один ICompiledArtifact создаёт несколько sessions; каждой session назначаются разные аргументы; запуски чередуются; повторный запуск первой session не должен получать state второй. После dispose запрещённый lifecycle path должен завершаться явной ошибкой.

Цель: проверить H4.

Набор: поддерживаемая арифметика с константами и ветвями; неподдерживаемая форма входа; искусственный failing pass в тестовой инфраструктуре.

Ожидания: Disabled не использует route; Require отклоняет unsupported; Prefer fallback-ит только известную несовместимость; internal defect становится error; Debug включает подробный report; каждый pass проходит verifier.

Цель: проверить, что external consumer компилируется против facade, а implementation closure остаётся runtime-only.

Процедура: pack UniversalToolchain.Wist; проверить ref/net10.0 и lib/net10.0 ; создать чистый consumer project; выполнить restore/build/run только через package source; убедиться, что внутренние types не доступны как публичный compile-time contract.

Цель: проверить наблюдаемость без утечки source/runtime values по умолчанию.

Процедура: выполнить CLI с --trace; проверить schema version, source length/hash, selected dialect/backend, coarse stages и result summary; убедиться, что raw formula/arguments отсутствуют в default artifact.

10.4. Verification record и текущий snapshot

В VERIFICATION.md зафиксирован baseline от 14 июля 2026 года:

- build 74 solution projects и standalone benchmark project;
- compile check 75 repository .csproj;
- тестовые сборки: 452, 288 и 585 тестов;
- всего 1325 тестов, 0 failed, 0 skipped.

Эта запись относится к зафиксированному тогда состоянию и не автоматически распространяется на текущий snapshot с 76 проектами. Перед подачей итоговой работы должен быть выполнен новый чистый прогон с обновлением commit/hash, recursive manifest, build/test/pack/docs evidence. В основной вывод не включаются неподтверждённые новые численные результаты.

11. Методика измерения производительности

11.1. Разделение фаз

В DSL runtime существуют принципиально разные операции:

1. создание engine и runtime composition;
2. lexer/parser/binding;
3. компиляция в backend artifact;
4. разовое Evaluate;
5. многократный вызов готового делегата.

Сравнение C# method invocation с WistEngine.Evaluate измеряет не только runtime overhead, но и подготовительные стадии. Сравнение заранее скомпилированного Wist delegate с evaluator, который парсит на каждом вызове, также некорректно. Поэтому проект разделяет три suite.

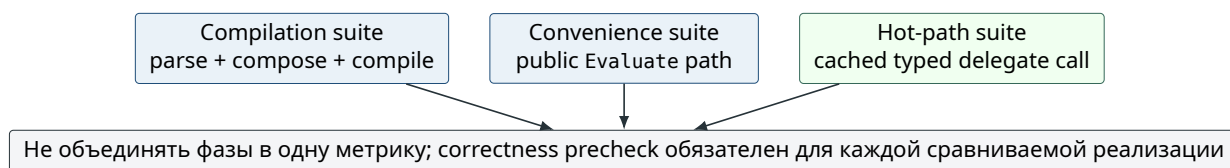


Рис. 7: Разделение benchmark-сценариев по фазам жизненного цикла

11.2. Compilation suite

Suite измеряет создание необходимого контекста и компиляцию формулы. Для анализа важны:

- mean/error/stddev;
- allocations;
- параметры выражения;
- backend и optimization policy;
- cold-start effects;
- raw BenchmarkDotNet artifacts.

Результат отвечает на вопрос: сколько стоит подготовить формулу к повторному использованию. Он не характеризует скорость дальнейших вызовов.

11.3. Convenience suite

Suite измеряет публичный Evaluate-путь. Это релевантно для одноразовых вычислений и прототипирования. В него входят проверки, подготовка

bindings и execution orchestration. Evaluate может быть медленнее compiled delegate и при этом оставаться корректным удобным API.

11.4. Hot-path suite

Suite измеряет только готовые артефакты. Для Wist это типизированный compiled delegate. Из измерения исключаются создание engine, parsing, composition и compilation. Сравниваемые альтернативы также должны быть заранее подготовлены. Перед benchmark выполняется parity precheck, подтверждающий одинаковое вычисление.

11.5. Evidence package и допустимые выводы

BenchmarkDotNet создаёт отдельное приложение для benchmark toolchain и публикует отчёты в BenchmarkDotNet.Artifacts [24, 25]. Для воспроизводимого утверждения недостаточно скриншота summary. Необходимы commit/hash, dotnet --info, CPU/OS, job configuration, raw CSV/Markdown/HTML, warnings, outliers и код подготовки сценария.

Без нового полного прогона текущей версии работа не публикует численную таблицу и не повторяет старые nanosecond-результаты как актуальные. Допустимое архитектурное утверждение уже сейчас: проект предоставляет compile-once/invoke-many путь и отделяет его измерение от cold/convenience phases. Утверждение о конкретном коэффициенте производительности требует нового evidence package.

12. Текущее состояние, ограничения и направления развития

Текущий snapshot подтверждает наличие многоуровневого pipeline, runtime manifests, capability contracts, Wist facade, typed compiled programs, structured diagnostics, trace v2 и экспериментальных SSA-компонентов. Архитектура уже содержит необходимые границы для расширения, но зрелость разных слоёв неодинакова: Wist-first путь существенно лучше проработан, чем независимое использование framework другим языком; SSA coverage ограничен; Bytecode tags требуют дальнейшей формализации; process-level sandbox отсутствует; полный trace внутренних представлений ещё не реализован.

Следующая таблица объединяет текущие ограничения и требуемое продолжение без повторения одних и тех же caveats в нескольких разделах.

Область	Текущее состояние	Следующий проверяемый шаг
Reproducibility	предыдущий verification baseline не совпадает с текущим snapshot	clean unpack, manifest regeneration, build/test/pack/docs и сохранение raw logs
Framework generality	основной consumer — Wist	независимый минимальный DSL consumer без Wist-specific branches
Bytecode contracts	часть семантики выражена tags	централизованный registry, producer/consumer matrix и verifier
Trace	versioned redacted coarse trace	стабильные snapshots стадий AST, Bytecode, AIR, SSA и backend
SSA	opt-in route для ограниченного shape	differential corpus, unsupported-shape taxonomy и pass-level evidence
Security	restricted surface и preflight limits	process isolation guidance, time/memory quotas и provider hardening
Performance	suites разделены по фазам	clean BenchmarkDotNet run с raw artifacts и correctness precheck

13. Личный вклад автора

Автором выполнены следующие группы работ:

1. сформулирована задача управляемого встраиваемого DSL runtime для .NET;
2. спроектирована многостадийная архитектура Source, AST, Bytecode, AIR и Backend;
3. разработан и развит эталонный язык Wist;
4. реализована модульная модель frontend-расширений;
5. разработана диалектная композиция с build plan и manifest-backed selection;
6. реализованы interpreter и CIL/DynamicMethod execution paths;
7. выделена backend-neutral граница compiled artifact/session;
8. разработан публичный пакет UniversalToolchain.Wist и typed facade;
9. реализованы restricted profiles, structured diagnostics и preflight limits;
10. сформулирована модель безопасности, отделяющая language restriction от sandbox;
11. удалён устаревший LogsViewer и введён versioned trace artifact;
12. реализован экспериментальный SSA route с lowering, passes, emission и

verifiers;

13. создана тестовая стратегия, включающая parity и отрицательные архитектурные проверки;
14. benchmark suite разделён по фазам исполнения;
15. подготовлена документация для пользователей, авторов модулей и разработчиков framework.

Сторонние инструменты использованы как инфраструктура: .NET SDK и CLR — платформа, BenchmarkDotNet — измерительная система, GitHub — version control/CI, NuGet — доставка пакета. Сравнимые библиотеки и compiler frameworks не входят в авторскую реализацию.

14. Заключение

UniversalToolchain решает не задачу «вычислить ещё одно выражение», а задачу владения расширяемым DSL runtime. Его ключевые свойства определяются границами: frontend не знает backend, Bytecode и AIR имеют разные роли, диалект выбирает runtime до activation, capability подтверждает применимость target-specific преобразования, verifier останавливает некорректный IR, а public facade не раскрывает внутренний backend artifact.

Wist показывает, что эти границы можно собрать в работающий .NET-путь: исходный текст проходит lexer/parser/AST, Bytecode и AIR; программа исполняется интерпретатором или понижается в CIL; restricted preset предоставляет типизированную интеграцию; SSA добавляется как контролируемый эксперимент, а не как обязательная перестройка системы.

Главная незавершённая работа относится не к добавлению случайных языковых возможностей, а к усилению доказательной базы: clean reproducibility, независимый framework consumer, bytecode verifier, полный trace и differential corpus для SSA/parity. После этих шагов архитектурные утверждения можно связывать не только с кодом и тестовой стратегией, но и с воспроизводимым evidence package текущей версии.

Приложение А. Команды воспроизведения

Ниже приведён целевой сценарий чистой проверки. Перед выпуском отчёта необходимо заменить <ARTICLE_COMMIT> на фактический commit/tag и сохранить stdout/stderr каждой обязательной команды.

А.1. Идентичность исходного дерева

```
1 git clone https://github.com/Misha1302/Wist2.git
2 cd Wist2
3 git checkout <ARTICLE_COMMIT>
4 git rev-parse HEAD
5 sha256sum -c MANIFEST.sha256
```

A.2. Окружение и каноническая проверка

```
1 dotnet --info
2 node --version
3 npm --version
4 python3 --version
5
6 ./build.sh
```

build.sh должен завершать restore, build, тестовые наборы, pack, package-surface checks и документационные проверки согласно текущему repository contract. Если скрипт меняет состав шагов, отчёт обязан ссылаться на конкретную версию скрипта.

A.3. Публичный sample и CLI trace

```
1 dotnet run -c Release --no-build --no-restore \
2   --project samples/Wist.RolloutScoring/Wist.RolloutScoring.csproj
3
4 dotnet run -c Release --no-build --no-restore \
5   --project UniversalToolchain/Wistc/Wistc.csproj -- \
6   run --eval "(2 + 2) * 3" --backend compiler --trace trace.json
```

После CLI-запуска проверяются schema version, selected dialect/backend, coarse stage sequence и отсутствие raw source/arguments в trace по умолчанию.

A.4. Package surface и clean consumer

```
1 python3 Tools/check-wist-package-surface.py artifacts/packages/*.nupkg
2 ./Tools/smoke-published-wist-package.sh artifacts/packages/*.nupkg
```

Проверка считается завершённой только после restore/build/run отдельного consumer project, использующего .nupkg как внешний package source.

A.5. Измерения производительности

```
1 find UniversalToolchain -name '*Benchmark*.csproj' -print
2
3 dotnet run -c Release --project <benchmark.csproj> -- \
4   --filter '*FormulaCompilationBenchmarks*'
5
6 dotnet run -c Release --project <benchmark.csproj> -- \
7   --filter '*FormulaConvenienceBenchmarks*'
8
9 dotnet run -c Release --project <benchmark.csproj> -- \
10  --filter '*FormulaHotPathBenchmarks*'
```


Перед каждым benchmark выполняется correctness precheck. В evidence package сохраняются commit, dotnet --info, BenchmarkDotNet job configuration, warnings и raw CSV/Markdown/HTML artifacts.

Приложение Б. Матрица доказательств

Утверждение	Артефакт	Проверка	Статус в текущем тексте
Restricted formula принимается	sample RolloutScoring	Validate, Compile и invoke	подтверждено исходным кодом; требуется новый clean run
Statement-style rule отклоняется	negative sample/test	structured validation result	реализовано и документировано
Composition предпешествует host activation	workflow/build plan/resolver	unit/integration tests	подтверждено архитектурой и recorded tests
Interpreter/CIL имеют общий API	artifact/session contracts	parity suite	реализовано; coverage ограничено shared subset
SSA route verifier-gated	Ssa projects + report	route tests	реализовано как experimental
NuGet facade ограничен	package csproj + surface checker	pack/consumer smoke	заявлено в build; новый snapshot smoke не выполнен
Текущий snapshot release-clean	MANIFEST + clean build	manifest, build, test, pack и docs	не подтверждено: 5 manifest mismatches
Количественная производительность	BDN raw artifacts	три suites + parity precheck	не заявляется без нового прогона

Приложение В. Термины

Диалект — выбранный профиль языка и runtime: модули, бэкенды, оптимизаторы и capabilities.

Runtime manifest — декларативное описание runtime-компонента, используемое resolver-ом до активации.

DialectBuildPlan — упорядоченный результат компиляции dialect source, содержащий требования к составу среды.

SelectedRuntimePlan — разрешённый фактический набор компонентов, на основе которого создаётся host.

Semantic parity — требование одинакового наблюдаемого поведения раз-

ных backend-путей на общем поддерживаемом подмножестве.

Capability — машинно проверяемое утверждение о поддержке операции или контракта выбранным runtime/backend.

Compiled artifact — неизменяемый результат компиляции, не зависящий на внешней границе от конкретного backend-типа.

Execution session — объект конкретного запуска compiled artifact, владеющий аргументами и изменяемым execution state.

Intrinsic — операция IR, которую backend может реализовать специальным способом; её использование требует capability.

Verifier-gated route — маршрут преобразования, где вход и каждый промежуточный результат проверяются до продолжения.

Controlled fallback — явно разрешённый возврат к базовому пути только для классифицированной несовместимости, а не для произвольного internal exception.

Приложение Г. Публичные контракты Wist facade

Г.1. WistEngineOptions

Параметры создания engine образуют конфигурационную границу и копируются в snapshot. Ключевые группы:

Группа	Назначение	Архитектурное ограничение
Preset / DialectSource	выбрать поставляемый или пользовательский dialect	одновременно должен существовать однозначный source of truth
Backend / BackendAlias	выбрать execution path	выбранный backend обязан входить в selected runtime plan
AllowedAssemblies	ограничить managed-call resolution	allowlist не заменяет method/provider policy
ResourceLimits	preflight-проверки	значения валидируются до создания host
Optimization	включить SSA policy и diagnostics	default не должен неожиданно активировать experimental route

Options snapshot предотвращает time-of-check/time-of-use расхождение: вызывающая сторона не может изменить коллекцию allowed assemblies после создания engine и тем самым неявно расширить уже скомпонованную среду.

Г.2. WistValidationResult

Validation result содержит флаг успешности, основное сообщение, structured diagnostics и optimization report. Контракт предназначен для ожидаемого внешнего ввода: UI может показать пользователю diagnostic code/stage/span без перехвата внутренних exception types.

Успешная validation доказывает, что формула может быть скомпилирована для выбранного профиля и объявленных типов параметров. Она не доказывает корректность бизнес-смысла формулы и не выполняет действие хоста.

Г.3. WistProgram

WistProgram<TDelegate> связывает:

- исходную или нормализованную metadata программы;
- типизированный CompiledDelegate;
- сведения о параметрах;
- optimization report.

Generic type является частью пользовательского контракта. Ошибка количества или типов parameter names должна обнаруживаться до выдачи программы, а не при первом произвольном вызове.

Г.4. WistCompileResult

Non-throwing result различает успешную typed program и диагностированный failure. Он полезен для систем, где формулу редактирует пользователь и invalid input является нормальным состоянием workflow. Internal defects при этом не должны безусловно нормализоваться до той же категории, что syntax error.

Г.5. WistOptimizationReport

Report является наблюдаемым доказательством маршрута, а не обещанием скорости. Значимые поля:

- Used — был ли применён SSA route;
- Fallback — произошёл ли разрешённый возврат;
- Profile — выбранная политика/профиль;
- InputAirInstructionCount и OutputAirInstructionCount;
- ExecutedPasses;
- Diagnostics;
- Trace для debug mode.

Приложение может логировать report для диагностики rollout, но не должно строить security decision только по факту Used=true.

Приложение Д. Диалектные сценарии

Д.1. Минимальная арифметика

Минимальный профиль должен включать только компоненты, необходимые для чисел, пробелов, `score` и `arithmetic lowering`, а также явно выбранный `backend`. Его задача — продемонстрировать, что язык может быть собран из подмножества модулей, а не всегда активируется как монолитный `full Wist`.

Д.2. Pricing-restricted

`Pricing profile` разрешает формулу над `declared bindings` и отклоняет `statement-style declarations`. Сценарий проверяет не предметную сложность расчёта, а три архитектурных свойства:

1. разрешённая формула проходит тот же общий `pipeline`;
2. запрещённая возможность отсутствует в `selected language surface`;
3. отказ происходит до исполнения и представлен диагностикой.

Д.3. Function-calls-safe-math

Профиль безопасных математических функций демонстрирует `managed-call boundary`. Функции должны быть представлены через нейтральную `callable/runtime` модель, а разрешение ограничено выбранным `policy` выбранных `provider` и `assembly`. Этот пример не является общим `type-directed overload resolver` и не должен описываться как полная поддержка `C# method calls`.

Д.4. SSA profile

`SSA profile` включает `opt-in optimizer route`. Его назначение — проверка `composition`, `route observability` и сохранения семантики поддерживаемого подмножества. Он не меняет публичную модель языка и не превращает `SSA` в самостоятельный `backend`. При `Require` неподдерживаемый `shape` обязан завершаться ошибкой; при `Prefer` разрешён только классифицированный `fallback`.

Д.5. Composition-restricted

`Composition-restricted profile` служит для проверки собственно механизма выбора компонентов: присутствие компонента в `repository catalog` не означает его доступность в `runtime`. Такой профиль полезен как отрицательный `oracle` для тестов `activation` и `capability leakage`.

Список литературы

- [1] Martin Fowler. *Domain-Specific Languages*. Addison-Wesley, 2010.
- [2] Markus Völter, Sebastian Benz, Christian Dietrich, Birgit Engelmänn, Mats Helander, Lennart C. L. Kats, Eelco Visser и Guido Wachsmuth. *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. 2013. URL: <https://dslbook.org/> (дата обр. 20.07.2026).
- [3] Mikhail Razakov. *UniversalToolchain/Wist2: Source Code, Documentation and Verification Records*. URL: <https://github.com/Misha1302/Wist2> (дата обр. 20.07.2026).
- [4] Microsoft. *.NET Compiler Platform SDK*. URL: <https://learn.microsoft.com/dotnet/csharp/roslyn-sdk/> (дата обр. 20.07.2026).
- [5] Microsoft. *.NET Compiler Platform SDK Concepts and Object Model*. URL: <https://learn.microsoft.com/dotnet/csharp/roslyn-sdk/compiler-api-model> (дата обр. 20.07.2026).
- [6] NCalc Project. *NCalc Documentation*. URL: <https://ncalc.github.io/ncalc/> (дата обр. 20.07.2026).
- [7] Dynamic Expresso Project. *Dynamic Expresso: C# Expressions Interpreter*. URL: <https://github.com/dynamicexpresso/DynamicExpresso> (дата обр. 20.07.2026).
- [8] Microsoft. *RulesEngine*. URL: <https://github.com/microsoft/RulesEngine> (дата обр. 20.07.2026).
- [9] ANTLR Project. *ANTLR Parser Generator*. URL: <https://www.antlr.org/> (дата обр. 20.07.2026).
- [10] Terence Parr и Kathleen Fisher. «LL(*): The Foundation of the ANTLR Parser Generator». В: *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. 2011, с. 425—436. DOI: 10.1145/1993498.1993548.
- [11] JetBrains. *MPS Documentation*. URL: <https://www.jetbrains.com/help/mps/> (дата обр. 20.07.2026).
- [12] Chris Lattner, Mehdi Amini, Uday Bondhugula и др. «MLIR: A Compiler Infrastructure for the End of Moore’s Law». В: (2020). arXiv: 2002.11054 [cs.PL]. URL: <https://arxiv.org/abs/2002.11054>.
- [13] LLVM Project. *MLIR Language Reference and Rationale*. URL: <https://mlir.llvm.org/docs/> (дата обр. 20.07.2026).
- [14] Christian Wimmer и Thomas Würthinger. «Truffle: A Self-Optimizing Runtime System». В: *Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity*. 2012, с. 13—14. DOI: 10.1145/2384716.2384723.
- [15] Alfred V. Aho, Monica S. Lam, Ravi Sethi и Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. 2-е изд. Pearson, 2006.
- [16] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [17] Cliff Click и Michael Paleczny. «A Simple Graph-Based Intermediate Representation». В: *Proceedings of the ACM SIGPLAN Workshop on Intermediate Representations*. 1995, с. 35—49.
- [18] Michel Steuwer, Thomas Köhler, Bastian Köpcke и Federico Pizzuti. «RISE & Shine: Language-Oriented Compiler Design». В: (2022). arXiv: 2201.03611 [cs.PL]. URL: <https://arxiv.org/abs/2201.03611>.

- [19] Ecma International. *ECMA-335: Common Language Infrastructure*. 6-е изд. 2012. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-335/> (дата обр. 20.07.2026).
- [20] Microsoft. *System.Reflection.Emit.DynamicMethod Class*. URL: <https://learn.microsoft.com/dotnet/api/system.reflection.emit.dynamicmethod> (дата обр. 20.07.2026).
- [21] Microsoft. *DynamicMethod.Invoke Supplementary Remarks*. URL: <https://learn.microsoft.com/dotnet/fundamentals/runtime-libraries/system-reflection-emit-dynamicmethod-invoke> (дата обр. 20.07.2026).
- [22] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman и F. Kenneth Zadeck. «Efficiently Computing Static Single Assignment Form and the Control Dependence Graph». В: *ACM Transactions on Programming Languages and Systems* 13.4 (1991), с. 451—490. DOI: 10.1145/115372.115320.
- [23] Mark N. Wegman и F. Kenneth Zadeck. «Constant Propagation with Conditional Branches». В: *ACM Transactions on Programming Languages and Systems* 13.2 (1991), с. 181—210. DOI: 10.1145/103135.103136.
- [24] BenchmarkDotNet Project. *Getting Started*. URL: <https://benchmarkdotnet.org/articles/guides/getting-started.html> (дата обр. 20.07.2026).
- [25] BenchmarkDotNet Project. *Toolchains and Exporters*. URL: <https://benchmarkdotnet.org/articles/configs/> (дата обр. 20.07.2026).